Design Document: Async Credit Service

Exercise: Basta Tech Lead take-home **Stack:** FastAPI · Celery · RabbitMQ · MongoDB

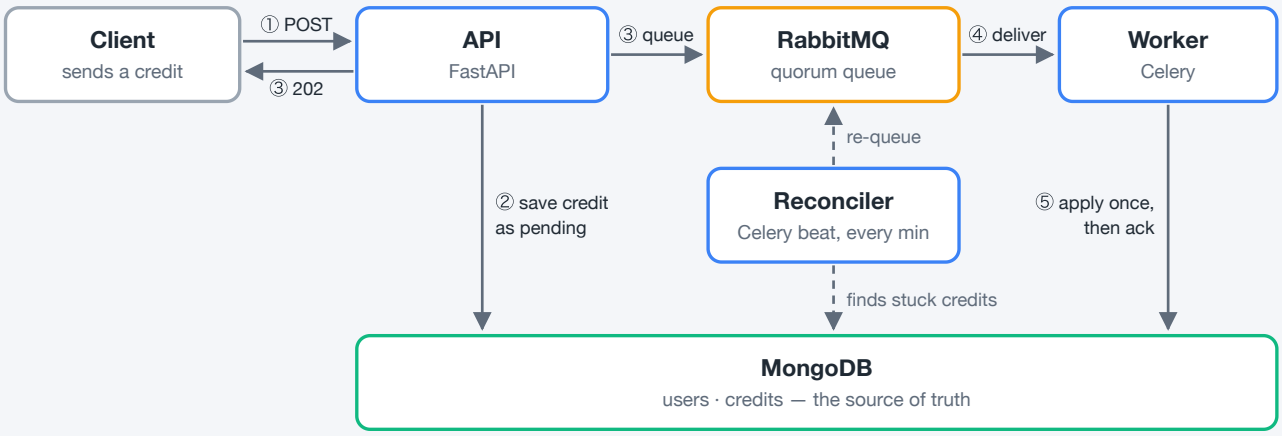
1. Goal

Credit a user's balance asynchronously: the API answers at once, and a worker applies the credit later. **FastAPI** validates and saves each credit, **RabbitMQ** holds it until a **Celery** worker applies it, and **MongoDB** stores every credit and balance as the source of truth. The rule that matters most: **every accepted credit is applied exactly once, or fails without touching the balance, even when a process crashes or a request is repeated.**

2. Assumptions and scope

- **Credits only add money.** No debits, amounts are positive, 2 decimal places, and capped at 1,000,000. One currency.
- **Users must already exist** (`404` otherwise). Creating them is another service's job; a seed command makes test users.
- **User status follows `pending_credits`.** Read literally, the brief would mark a user `processed` while a second credit is still pending.
- **No authentication or rate limiting.** Only local demo. We would add both for a real deployment.

3. Architecture & Data Flow



One Docker image runs the API, worker and reconciler. Tasks carry only a credit ID; MongoDB holds everything else.

1. **POST.** The client sends `POST /credits` with a required `Idempotency-Key`.
2. **Save.** In one MongoDB transaction, the API inserts the credit as `pending` and adds 1 to the user's `pending_credits` (status `processing`).
3. **Queue.** Only after the commit, it publishes the credit ID, waits for RabbitMQ's confirm, and answers `202`.
4. **Deliver.** RabbitMQ hands the task to a worker.
5. **Apply.** In one transaction, the worker moves the credit to `applied` *only if still pending*, adds the amount to `balance`, and subtracts 1 from `pending_credits` (status `processed` at zero). Then it acknowledges the task.

4. Data

Collection	Fields
users	`user_id` (unique) · `balance` (`Decimal128`), created by the first credit's `$inc` · `status` and `pending_credits`: `processed` only when no credit is in flight · `created_at`, `updated_at`
credits	`_id`: the credit ID returned to the client, also the Celery task ID · `amount`: `Decimal128`, never a float · `status`: `pending` → `applied` or `failed` · `idempotency_key` (unique) · `attempts`: reconciler re-queues, `failed` after 3

5. Key decisions and trade-offs

Save first, then queue. Publishing first could queue a credit whose save then fails. Saving first means a failed publish leaves a `pending` credit, which the reconciler re-queues after 5 minutes.

Exactly once on top of at-least-once delivery. The worker's "only if still pending" update runs in the same transaction as the balance change, and the task is acknowledged only after commit. A duplicate delivery does nothing; a crash mid-task leaves the task in RabbitMQ for another worker.

The Idempotency-Key is required. Without one, a retry after a lost response looks like a new credit. Missing key: `400`. Same key, different body: `422`. A concurrent repeat waits for the original to commit and gets it back. The cost: every client must send a key.

RabbitMQ, not Redis. Messages on disk, confirmed publishes, and a built-in delivery limit and dead-letter queue. The cost: one more service to run.

A reconciler, not a transactional outbox. It covers the gap between saving and queueing with less machinery. The cost: up to about 6 minutes' delay when RabbitMQ is down. It's the first thing to change if that matters.

6. Failure handling

Failure	What happens — handled · retried · error or `failed`
Response lost, client retries	The same key returns the original credit. One credit.
MongoDB down for the API	`503` within 5 s; the client retries with the same key.
RabbitMQ down at publish	Still `202`; the reconciler queues the credit later.
Worker killed mid-credit	Nothing committed; RabbitMQ redelivers. Applied once.
MongoDB down for the worker	Retries after 1, 2, 4 … 64 s (127 s), then `failed`, balance untouched.
Task keeps crashing workers	Dead-letter queue after 6 deliveries; the reconciler retries 3 times, then `failed`.

> **Found by the chaos tests**
> A killed worker leaves its MongoDB transaction open for up to 90 s (measured: 88 s), blocking the retried credit; the worker's retries now last 127 s. A silently dead connection hung the API for minutes, which led to a 5-second limit on every database call and publish.

7. Testing strategy

Kind	What it proves	Run
Unit	Amount parsing, exact decimals in JSON, log format.	`uv run pytest` 65 tests, ~35 s, CI on every push
Integration	API and worker against real MongoDB and RabbitMQ: validation, idempotency, duplicate delivery, retries, dead-lettering, reconciler.	
Chaos	Toxiproxy drops and freezes connections; processes are killed mid-credit. Each failure above has a test.	`pytest -m chaos` 10 tests, ~3 min
Load	Thousands of concurrent credits, 10% repeated; every balance must equal the sum of its credits.	`load_test.py` ~5 s

Real MongoDB and RabbitMQ, not mocks: slower, but that is how the two bugs above were found.

8. Next steps

- **Authentication and rate limiting:** first, before anyone else can reach the API.
- **Scaling:** one API process handles about 440 credits/s and is the bottleneck; they share nothing, so add more. Credits for one user take turns, which keeps balances exact. MongoDB is one node here; production needs three.
- **Alerts (not built):** page on dead-lettered messages, `failed` credits, a credit `pending` over 10 minutes, or a growing queue; warn on slow or failing responses.